Министерство образования Республики Беларусь
Учреждение образования «Белорусский государственный университет информатики и радиоэлектроники»

Факультет компьютерных систем и сетей
Кафедра информатики
Дисциплина «Методы оптимизации и управления»

ОТЧЕТ
к лабораторной работе №4
на тему:
«НАЧАЛЬНАЯ ФАЗА СИМПЛЕКС-МЕТОДА»
БГУИР 6-05-0612-02 67

Выполнил студент группы 353503
КОХАН Артём Игоревич

(дата, подпись студента)

Проверил доцент каф. ИКТ
ГОРБУКОВ Дмитрий Михайлович

(дата, подпись преподавателя)

Минск 2026

СОДЕРЖАНИЕ

1 Индивидуальное задание 3
2 Выполнение работы 4
Вывод 6
Список использованных источников 7
Приложение А (справочное) Исходный код 8

1 ИНДИВИДУАЛЬНОЕ ЗАДАНИЕ

Реализовать начальную фазу симплекс-метода для задачи линейного программирования в канонической форме. По заданным векторам c и b и матрице A требуется определить, совместна ли задача, и в случае положительного ответа построить некоторый базисный допустимый план.

При выполнении лабораторной работы необходимо учитывать алгоритм построения вспомогательной задачи, проверку значений искусственных переменных и процедуру удаления искусственных переменных из базиса в соответствии с условием лабораторной работы.

2 ВЫПОЛНЕНИЕ РАБОТЫ

В программе реализованы вспомогательные операции линейной алгебры без использования внешних библиотек, функция обновления обратной матрицы базиса по методу из первой лабораторной работы, основная фаза симплекс-метода и отдельная процедура начальной фазы.

На первом этапе программа приводит вектор правых частей к неотрицательному виду, затем строит вспомогательную задачу с искусственными переменными и начальным базисом B = [4, 5]. Для примера из методических указаний формируются c_tilde = [0, 0, 0, -1, -1], матрица A_tilde = [[1, 1, 1, 1, 0], [2, 2, 2, 0, 1]] и начальный план x_tilde = [0, 0, 0, 0, 0].

На первой итерации основной фазы получены вектор потенциалов u = [-1, -1], вектор оценок delta = [3, 3, 3, 0, 0], входящая переменная x1, направление z = [1, 2] и шаг theta0 = 0. После замены базиса B становится равным [1, 5]. На второй итерации все оценки оказываются неположительными, поэтому оптимальный план вспомогательной задачи равен x_tilde* = [0, 0, 0, 0, 0].

Так как значения всех искусственных переменных равны нулю, исходная задача совместна. Далее выполняется очистка базиса от искусственных переменных. Для искусственной переменной x5 подходящей замены среди небазисных исходных переменных не найдено, поэтому второе ограничение распознается как линейно зависимое и удаляется.

В результате работы алгоритма получен базисный допустимый план исходной задачи x = [0, 0, 0], базис B = [1], итоговая матрица ограничений A = [[1, 1, 1]] и вектор правых частей b = [0]. Результат работы программы приведен ниже на рисунке 2.1.

```
Шаг 3. Решаем вспомогательную задачу основной фазой.

--- Итерация 1 ---
Текущий план x = [0.0, 0.0, 0.0, 0.0, 0.0]
Текущий базис B = [4, 5]
Вектор потенциалов u = [-1.0, -1.0]
Вектор оценок delta = [3.0, 3.0, 3.0, 0.0, 0.0]
В базис входит x1
Вектор z = [1.0, 2.0]
Вектор theta = [0.0, 0.0]
Минимальный шаг theta0 = 0.0
Базис покидает x4

--- Итерация 2 ---
Текущий план x = [0.0, 0.0, 0.0, 0.0, 0.0]
Текущий базис B = [1, 5]
Вектор потенциалов u = [2.0, -1.0]
Вектор оценок delta = [0.0, 0.0, 0.0, -3.0, 0.0]
Все оценки неположительны. Найден оптимальный план.

Оптимальный план вспомогательной задачи:
x_tilde* = [0.0, 0.0, 0.0, 0.0, 0.0]
Искусственные переменные = [0.0, 0.0]

Шаг 4. Удаляем искусственные переменные из базиса.

Удаляем искусственную переменную x5 из базиса.
Она соответствует ограничению 2.
l(2) = [1.0, 0.0]
l(3) = [1.0, 0.0]
Подходящей замены нет. Удаляем линейно зависимое ограничение 2.

--- Результат ---
Базисный допустимый план x = [0.0, 0.0, 0.0]
Базис B = [1]
Итоговая матрица A =
[1.0, 1.0, 1.0]
Итоговый вектор b = [0.0]
```

Рисунок 2.1 – Результат работы программы

ВЫВОД

В ходе лабораторной работы реализована начальная фаза симплекс-метода, позволяющая определить совместность задачи линейного программирования и построить базисный допустимый план. На тестовом примере из методических указаний алгоритм корректно получил нулевой допустимый план, обнулил искусственные переменные и удалил линейно зависимое ограничение. Полученные результаты согласуются с условием лабораторной работы, что подтверждает корректность реализации.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

[1] Python Documentation [Электронный ресурс]. – Режим доступа: https://docs.python.org/. – Дата доступа: 09.04.2026.

[2] NumPy Documentation [Электронный ресурс]. – Режим доступа: https://numpy.org/doc/. – Дата доступа: 09.04.2026.

ПРИЛОЖЕНИЕ А
(справочное)
Исходный код

Листинг А.1 – Реализация начальной фазы симплекс-метода

```
TOL = 1e-9

def copy_matrix(matrix):
    return [row[:] for row in matrix]


def identity_matrix(n):
    return [[1.0 if i == j else 0.0 for j in range(n)] for i in
range(n)]


def transpose(matrix):
    return [list(row) for row in zip(*matrix)]


def matrix_vector_mul(matrix, vector):
    return [sum(matrix[i][j] * vector[j] for j in range(len(vector)))
for i in range(len(matrix))]


def row_vector_matrix_mul(vector, matrix):
    cols = len(matrix[0])
    rows = len(matrix)
    return [sum(vector[i] * matrix[i][j] for i in range(rows)) for j
in range(cols)]


def matrix_mul(a, b):
    rows = len(a)
    cols = len(b[0])
    inner = len(b)
    result = [[0.0 for _ in range(cols)] for _ in range(rows)]
    for i in range(rows):
        for k in range(inner):
            if abs(a[i][k]) < TOL:
                continue
            for j in range(cols):
                result[i][j] += a[i][k] * b[k][j]
    return result


def inverse_matrix(matrix):
    n = len(matrix)
    aug = [matrix[i][:] + identity_matrix(n)[i] for i in range(n)]

    for col in range(n):
        pivot = col
        while pivot < n and abs(aug[pivot][col]) < TOL:
            pivot += 1
        if pivot == n:
```

```
            raise ValueError("Матрица вырождена.")
        if pivot != col:
            aug[col], aug[pivot] = aug[pivot], aug[col]

        pivot_value = aug[col][col]
        for j in range(2 * n):
            aug[col][j] /= pivot_value

        for row in range(n):
            if row == col:
                continue
            factor = aug[row][col]
            if abs(factor) < TOL:
                continue
            for j in range(2 * n):
                aug[row][j] -= factor * aug[col][j]

    return [row[n:] for row in aug]


def get_submatrix_columns(matrix, columns):
    return [[row[j] for j in columns] for row in matrix]


def get_vector_by_indices(vector, indices):
    return [vector[i] for i in indices]


def concat_matrices_horizontally(left, right):
    return [left[i][:] + right[i][:] for i in range(len(left))]


def round_vector(vector, digits=6):
    return [round(x, digits) for x in vector]


def round_matrix(matrix, digits=6):
    return [round_vector(row, digits) for row in matrix]


def print_vector(name, vector):
    print(f"{name} = {round_vector(vector)}")


def print_matrix(name, matrix):
    print(f"{name} =")
    for row in round_matrix(matrix):
        print(row)


def update_inverse_matrix(a_b_inv, new_col, leaving_col_idx):
    """
    Обновление обратной матрицы базиса по методу из lab1.
    """
    l = matrix_vector_mul(a_b_inv, new_col)
    if abs(l[leaving_col_idx]) < TOL:
        raise ValueError("Невозможно обновить обратную матрицу
базиса.")
```

```
    l_tilde = l[:]
    l_tilde[leaving_col_idx] = -1.0
    factor = -1.0 / l[leaving_col_idx]
    l_hat = [factor * value for value in l_tilde]

    q = identity_matrix(len(a_b_inv))
    for row in range(len(q)):
        q[row][leaving_col_idx] = l_hat[row]

    return matrix_mul(q, a_b_inv)


def simplex_main_phase(a, c, x, basis, verbose=True):
    """
    Основная фаза симплекс-метода.

    Returns:
        (x_opt, basis_opt) или (None, None), если задача не ограничена
сверху.
    """
    m = len(a)
    x = x[:]
    basis = basis[:]
    a_b_inv = inverse_matrix(get_submatrix_columns(a, basis))

    iteration = 1
    while True:
        if verbose:
            print(f"\n--- Итерация {iteration} ---")
            print_vector("Текущий план x", x)
            print(f"Текущий базис B = {[idx + 1 for idx in basis]}")

        c_b = get_vector_by_indices(c, basis)
        u = row_vector_matrix_mul(c_b, a_b_inv)
        u_a = row_vector_matrix_mul(u, a)
        delta = [c[j] - u_a[j] for j in range(len(c))]

        if verbose:
            print_vector("Вектор потенциалов u", u)
            print_vector("Вектор оценок delta", delta)

        positive = [j for j, value in enumerate(delta) if value > TOL]
        if not positive:
            if verbose:
                print("Все оценки неположительны. Найден оптимальный
план.")
            return x, basis

        j0 = positive[0]
        entering_col = [a[i][j0] for i in range(m)]
        z = matrix_vector_mul(a_b_inv, entering_col)

        if verbose:
            print(f"В базис входит x{j0 + 1}")
            print_vector("Вектор z", z)

        theta = [float("inf")] * m
```

```
        for i in range(m):
            if z[i] > TOL:
                theta[i] = x[basis[i]] / z[i]

        theta0 = min(theta)
        if theta0 == float("inf"):
            if verbose:
                print("Целевая функция не ограничена сверху.")
            return None, None

        k = theta.index(theta0)
        leaving = basis[k]

        if verbose:
            print_vector("Вектор theta", theta)
            print(f"Минимальный шаг theta0 = {round(theta0, 6)}")
            print(f"Базис покидает x{leaving + 1}")

        new_x = x[:]
        for i in range(m):
            new_x[basis[i]] -= theta0 * z[i]
            if abs(new_x[basis[i]]) < TOL:
                new_x[basis[i]] = 0.0
        new_x[j0] = theta0
        new_x[leaving] = 0.0

        a_b_inv = update_inverse_matrix(a_b_inv, entering_col, k)
        basis[k] = j0
        x = new_x
        iteration += 1


def remove_artificial_from_basis(a_aux, b, basis, n, verbose=True):
    """
    Удаление искусственных переменных из базиса.

    Returns:
        (a_reduced, b_reduced, basis_reduced)
    """
    a_work = copy_matrix(a_aux)
    b_work = b[:]
    basis_work = basis[:]

    while any(index >= n for index in basis_work):
        jk = max(index for index in basis_work if index >= n)
        k = basis_work.index(jk)
        i = jk - n

        if verbose:
            print(f"\nУдаляем искусственную переменную x{jk + 1} из
базиса.")
            print(f"Она соответствует ограничению {i + 1}.")

        a_b_inv = inverse_matrix(get_submatrix_columns(a_work,
basis_work))
        replacement_found = False

        for j in range(n):
```

```
            if j in basis_work:
                continue
            col_j = [a_work[row][j] for row in range(len(a_work))]
            l = matrix_vector_mul(a_b_inv, col_j)
            if verbose:
                print_vector(f"l({j + 1})", l)
            if abs(l[k]) > TOL:
                basis_work[k] = j
                replacement_found = True
                if verbose:
                    print(f"Заменяем x{jk + 1} на x{j + 1} в базисе.")
                break

        if replacement_found:
            continue

        if verbose:
            print(f"Подходящей замены нет. Удаляем линейно зависимое
ограничение {i + 1}.")

        del a_work[i]
        del b_work[i]
        for row in a_work:
            del row[jk]
        basis_work.pop(k)

        for idx in range(len(basis_work)):
            if basis_work[idx] > jk:
                basis_work[idx] -= 1

    return a_work, b_work, basis_work


def initial_simplex_phase(c, a, b, verbose=True):
    """
    Начальная фаза симплекс-метода.

    Returns:
        (x, basis, a_reduced, b_reduced) или (None, None, None, None),
        если задача несовместна.
    """
    a = copy_matrix(a)
    b = b[:]
    c = c[:]

    m = len(a)
    n = len(a[0])

    if verbose:
        print("--- Начальная фаза симплекс-метода ---")
        print("Шаг 1. Делаем вектор правых частей неотрицательным.")

    for i in range(m):
        if b[i] < 0:
            b[i] *= -1
            for j in range(n):
                a[i][j] *= -1
```

```
    if verbose:
        print_matrix("A после коррекции", a)
        print_vector("b после коррекции", b)

    if verbose:
        print("\nШаг 2. Строим вспомогательную задачу.")

    a_aux = concat_matrices_horizontally(a, identity_matrix(m))
    c_aux = [0.0] * n + [-1.0] * m
    x_aux = [0.0] * n + b[:]
    basis_aux = list(range(n, n + m))

    if verbose:
        print_vector("c_tilde", c_aux)
        print_matrix("A_tilde", a_aux)
        print_vector("Начальный план x_tilde", x_aux)
        print(f"Начальный базис B = {[idx + 1 for idx in basis_aux]}")

    if verbose:
        print("\nШаг 3. Решаем вспомогательную задачу основной
фазой.")

    x_aux_opt, basis_aux_opt = simplex_main_phase(a_aux, c_aux, x_aux,
basis_aux, verbose=verbose)
    if x_aux_opt is None:
        return None, None, None, None

    artificial_values = x_aux_opt[n:]
    if verbose:
        print("\nОптимальный план вспомогательной задачи:")
        print_vector("x_tilde*", x_aux_opt)
        print_vector("Искусственные переменные", artificial_values)

    if any(abs(value) > TOL for value in artificial_values):
        if verbose:
            print("Искусственные переменные не все равны нулю.
Исходная задача несовместна.")
        return None, None, None, None

    x = x_aux_opt[:n]
    x = [0.0 if abs(value) < TOL else value for value in x]

    if verbose:
        print("\nШаг 4. Удаляем искусственные переменные из базиса.")

    a_reduced, b_reduced, basis_reduced =
remove_artificial_from_basis(
        a_aux,
        b,
        basis_aux_opt,
        n,
        verbose=verbose,
    )

    a_original_reduced = [row[:n] for row in a_reduced]
    return x, basis_reduced, a_original_reduced, b_reduced
```

```
def print_result(x, basis, a, b):
    print("\n--- Результат ---")
    print_vector("Базисный допустимый план x", x)
    print(f"Базис B = {[idx + 1 for idx in basis]}")
    print_matrix("Итоговая матрица A", a)
    print_vector("Итоговый вектор b", b)


if __name__ == "__main__":
    # Пример из условия lab4_conditions.pdf
    c_example = [1.0, 0.0, 0.0]
    a_example = [
        [1.0, 1.0, 1.0],
        [2.0, 2.0, 2.0],
    ]
    b_example = [0.0, 0.0]

    x_result, basis_result, a_result, b_result =
initial_simplex_phase(
        c_example,
        a_example,
        b_example,
        verbose=True,
    )

    if x_result is None:
        print("\nОтвет: исходная задача не имеет допустимых планов.")
    else:
        print_result(x_result, basis_result, a_result, b_result)
```